ANSIBLE PLAYBOOKS

Complete Classroom Notes

Beginner to Intermediate • Theory + Practical Labs

What this guide covers

Playbook structure • YAML • Plays, tasks and modules • Variables • Apache deployment • Handlers • Execution options • Troubleshooting • Viva questions

Prepared for Linux and DevOps classroom teaching

Note: Examples use an AlmaLinux/Rocky Linux managed node unless stated otherwise. Replace sample IP addresses and usernames with your lab values.

Contents

• 1. What is an Ansible Playbook?

• 2. Why use playbooks?

• 3. Ad-hoc commands vs playbooks

• 4. YAML fundamentals

• 5. Playbook structure: play, task, module

• 6. Practical lab: first playbook

• 7. Practical lab: Apache website deployment

• 8. Common execution options

• 9. Variables in playbooks

• 10. Handlers and notify

• 11. Idempotency

• 12. Troubleshooting

• 13. Student exercises

• 14. Interview and viva questions

• 15. Quick revision sheet

Learning objectives

By the end of this module, students should be able to explain what a playbook is, read basic YAML, write a simple playbook, target inventory hosts, execute and validate a playbook, install Apache, and troubleshoot common errors.

1. What is an Ansible Playbook?

Definition: An Ansible Playbook is a YAML file that defines automation instructions to be executed on one or more managed hosts.

A playbook describes what tasks should be performed, where they should be performed, and how the target systems should be configured.

Malayalam / Manglish: Linux server-il manually cheyyunna tasks—package install cheyyuka, file copy cheyyuka, service start cheyyuka—ivayellam YAML file-il instructions aayi ezhuthi automatic aayi execute cheyyunnathaanu Ansible Playbook.

Real-time example

A company has 20 Linux servers. Apache must be installed, started, and configured with a company web page on all of them. Instead of repeating commands manually on each server, the administrator can write a playbook once and run it against the required inventory group.

Why do we use playbooks?

• **Automation:** Repetitive tasks can be executed automatically.

• **Consistency:** Apply the same intended configuration to many servers.

• **Time saving:** Reduce repeated manual commands.

• **Reusability:** Save instructions and run them again when needed.

• **Multi-server management:** Target a group of hosts in one run.

• **Documentation:** The YAML file records the intended configuration.

• **Idempotency:** Many Ansible modules avoid unnecessary changes when the desired state already exists.

2. Ad-hoc Command vs Playbook

Ad-hoc command	Playbook
Useful for a quick, one-off task.	Useful for repeatable workflows and multiple tasks.
Usually entered directly in the terminal.	Instructions are saved in a YAML file.
Convenient for quick checks or simple actions.	Easier to review, maintain, reuse, and version-control.
Example: ping a host.	Example: install Apache, publish a page, and start the service.

```
# Ad-hoc command
ansible webservers -m ansible.builtin.ping

# Playbook execution
ansible-playbook -i inventory.ini webserver.yml
```

3. YAML Fundamentals

YAML is a human-readable data serialization format. Ansible uses YAML to describe plays, tasks, variables, and module arguments.

```
student:
  name: Rahul
  course: Linux
  duration: 6
```

• A key and its value are separated by a colon and a space.

• Indentation shows the structure of nested data.

• Use spaces, not tabs, for indentation.

• List items begin with a hyphen followed by a space.

• Quote strings when necessary, especially values that could be interpreted as booleans, numbers, or special YAML syntax.

Common mistake: incorrect indentation can cause a YAML parsing error or change the meaning of the data.

4. Basic Playbook Structure

```
---
- name: Play description
  hosts: target_hosts
  become: true
  gather_facts: true

  vars:
    variable_name: value

  tasks:
    - name: First task
      ansible.builtin.debug:
        msg: "Hello from Ansible"
```

Component	Purpose
---	Indicates the start of a YAML document.

Component	Purpose
name	Descriptive name for a play or task.
hosts	Selects hosts or an inventory group.
become	Enables privilege escalation when required.
gather_facts	Controls collection of managed-host facts before tasks.
vars	Defines variables for the play.
tasks	Contains the ordered list of tasks.
Module	Performs an operation, such as copying a file or installing a package.

Play, task, and module

• **Playbook:** The complete YAML automation file.

• **Play:** Maps tasks and settings to selected hosts.

• **Task:** An individual unit of work.

• **Module:** The component that performs the operation.

Example: playbook = webserver.yml; play = configure web servers; tasks = install Apache, copy website, start service; modules = dnf, copy, service.

5. Practical Lab: First Playbook

Lab requirements

• Control node: Ubuntu with a supported Ansible installation.

• Managed node: AlmaLinux or Rocky Linux.

• Working SSH connectivity and suitable sudo permissions.

• An inventory file containing the managed host.

Step 1: Create a working directory

```
mkdir -p ~/ansible-lab
cd ~/ansible-lab
```

Run these commands on the control node. The mkdir command creates the lab directory; cd changes into it.

Step 2: Create inventory.ini

```
nano inventory.ini
```

Add the following example, replacing the sample IP address with your actual managed-node address:

```
[webservers]
webserver1 ansible_host=192.168.1.21 ansible_user=ansible
```

The group name is webservers. The inventory alias is webserver1. The ansible_host value specifies the actual address, and ansible_user specifies the SSH username.

Step 3: Test connectivity

```
ansible -i inventory.ini webservers -m ansible.builtin.ping
```

If SSH needs a password, add -k to prompt for it. If a task needs a sudo password, -K prompts for the privilege-escalation password.

Step 4: Create first-playbook.yml

```
nano first-playbook.yml
```

```
---
- name: My first Ansible playbook
  hosts: webservers
  gather_facts: false

  tasks:
    - name: Display a welcome message
      ansible.builtin.debug:
        msg: "Welcome to Ansible Playbooks!"
```

Step 5: Check syntax

```
ansible-playbook -i inventory.ini first-playbook.yml --syntax-check
```

Syntax-check checks whether Ansible can parse the playbook. It does not execute the tasks.

Step 6: Execute the playbook

```
ansible-playbook -i inventory.ini first-playbook.yml
```

Illustrative output:

```
PLAY [My first Ansible playbook]

TASK [Display a welcome message]
ok: [webserver1] => {
    "msg": "Welcome to Ansible Playbooks!"
}

PLAY RECAP
webserver1 : ok=1 changed=0 unreachable=0 failed=0
```

The exact output can vary by Ansible version and configuration. The example shows the general result, not a guarantee of your environment's output.

6. Practical Lab: Install Apache and Create a Website

This playbook targets AlmaLinux/Rocky Linux, where Apache is normally packaged as httpd.

```
---
- name: Configure Apache web server
  hosts: webservers
  become: true

  tasks:
    - name: Install Apache
      ansible.builtin.dnf:
        name: httpd
        state: present

    - name: Create a custom website page
      ansible.builtin.copy:
        dest: /var/www/html/index.html
        content: |
          <html>
          <head>
            <title>Student Portal</title>
          </head>
          <body>
            <h1>Welcome to Ansible Automation</h1>
            <p>This website is managed using Ansible.</p>
          </body>
          </html>
        owner: root
        group: root
        mode: '0644'
```

```
    - name: Start and enable Apache
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true
```

Run the playbook

```
ansible-playbook -i inventory.ini webserver.yml -K
```

The -K option asks for the become password when required. If the account has authorized passwordless sudo, the prompt may not be needed.

Understand the tasks

• **dnf:** Ensures the httpd package is installed.

• **copy:** Writes the supplied HTML content to /var/www/html/index.html.

• **dest:** Destination file path.

• **content:** Text to write to the file.

• **owner/group:** Sets file ownership.

• **mode: '0644':** Owner can read/write; group and others can read.

• **service:** Ensures the service is running and enabled at boot.

Verify on the managed server

```
systemctl status httpd
curl http://localhost/
```

To access the website from another computer, also verify network routing, firewall rules, and cloud security-group rules if applicable.

Ubuntu note: Ubuntu commonly uses the apt package manager and the apache2 package/service name. The playbook above is specifically for RHEL-family systems.

7. Important Playbook Execution Options

Command / option	Purpose
ansible-playbook -i inventory.ini webserver.yml	Execute the playbook.
--syntax-check	Check playbook syntax without running tasks.
--list-hosts	List hosts selected by the play.
--list-tasks	List tasks without executing them.
--check	Attempt a check-mode run where supported.
--diff	Show supported file/configuration differences.
--limit webserver1	Restrict execution to a selected host.
-K	Prompt for the become password.
-v	Show more execution details.

Check mode is not a perfect simulation of every task. Module support varies, and tasks that depend on earlier changes may behave differently from a real execution.

8. Variables in Playbooks

Variables store values that can be reused. They make playbooks easier to maintain and adapt.

```
---
- name: Install a package using a variable
  hosts: webservers
  become: true

  vars:
    package_name: httpd

  tasks:
    - name: Install the selected package
      ansible.builtin.dnf:
        name: "{{ package_name }}"
        state: present
```

Here, package_name is a variable. The expression {{ package_name }} references its value.

Malayalam: Oru value pala places-il use cheyyumbol oro place-ilum same value type cheyyanda. Variable-il store cheythu avashyamulla places-il reference cheyyam.

9. Handlers and notify

Handlers are special tasks that normally run when notified by another task that reports a change. They are commonly used to restart or reload a service after configuration changes.

```
---
- name: Manage Apache configuration
  hosts: webservers
  become: true

  tasks:
    - name: Update Apache configuration
      ansible.builtin.copy:
        src: files/httpd.conf
        dest: /etc/httpd/conf/httpd.conf
        owner: root
        group: root
        mode: '0644'
      notify: Restart Apache

  handlers:
    - name: Restart Apache
      ansible.builtin.service:
        name: httpd
        state: restarted
```

This example assumes files/httpd.conf exists and contains valid configuration for the target system. The handler is notified if the copy task reports a change. If the file is already identical, the copy task normally reports no change and does not notify the handler.

Malayalam: Configuration file change aayal mathram Apache restart cheyyan notify use cheyyunnu. Ith unnecessary restart avoid cheyyan help cheyyum.

10. Idempotency

Idempotency means that repeatedly applying the same desired configuration should leave the system in the same intended state, without unnecessary changes when it is already correct.

```
- name: Ensure Apache is installed
  ansible.builtin.dnf:
    name: httpd
    state: present
```

• First run: Apache may be installed, so the task reports changed.

• Later run: If Apache is already installed and no change is required, the task normally reports ok.

• Not every possible task is automatically idempotent; behavior depends on the module, arguments, and current system state.

11. Common Errors and Troubleshooting

Error	Possible cause	What to check
Syntax error	Invalid YAML or indentation	Review structure; run --syntax-check.
No hosts matched	Wrong group or host pattern	Check inventory and hosts: value.
UNREACHABLE	SSH/network problem	Check IP, SSH service, routing, and credentials.
Permission denied	Insufficient permissions	Check remote user and sudo configuration.
Missing sudo password	Become password required	Use -K or approved sudo configuration.
No package matching httpd	Wrong OS/package name	Verify distribution and package manager.
Undefined variable	Missing or misspelled variable	Check variable definitions and spelling.
Could not find or access	Wrong source path	Check file path relative to project.

Incorrect YAML example

```
tasks:
- name: Install Apache
  ansible.builtin.dnf:
  name: httpd
  state: present
```

Correct YAML example

```
tasks:
  - name: Install Apache
    ansible.builtin.dnf:
      name: httpd
      state: present
```

Module arguments must be indented beneath the module name. Use spaces rather than tabs.

12. Student Practical Exercises

• Create a playbook that displays the managed server's hostname.

• Create a playbook that installs Apache on AlmaLinux.

• Create a playbook that creates /opt/student-data.

• Create a playbook that copies an HTML page to /var/www/html/index.html.

• Create a playbook that starts and enables Apache.

• Use a variable for the package name.

• Install two packages using a playbook.

• Update an Apache configuration file and notify a handler.

• Run a playbook against only one selected host using --limit.

• Use syntax-check and check mode, then compare with a real execution.

Student submission requirements

• The YAML file.

• The execution command.

• Output or screenshots.

• A short explanation of each task.

• Validation command and result.

13. Interview and Viva Questions

1. What is an Ansible playbook?
Answer: A YAML file defining automation instructions for one or more managed hosts.

2. Which format is used for playbooks?
Answer: YAML.

3. What is a play?
Answer: A section mapping tasks and settings to selected hosts.

4. What is a task?
Answer: An individual unit of work executed by Ansible.

5. What is a module?
Answer: A reusable component that performs an operation, such as installing a package.

6. What does become: true do?
Answer: Enables privilege escalation, commonly through sudo, when required.

7. What does gather_facts control?
Answer: Whether Ansible collects information about managed hosts before tasks run.

8. What is --syntax-check used for?
Answer: Checking playbook syntax without executing tasks.

9. What is a handler?
Answer: A special task normally triggered by a notification.

10. What is idempotency?
Answer: Repeatedly applying a desired configuration without unnecessary changes when the state is already correct.

11. What is inventory?
Answer: The list or structure describing managed hosts and groups.

12. Why is indentation important in YAML?
Answer: Indentation represents nested structure and affects parsing.

13. What does the copy module do?
Answer: Copies content or files to a managed host.

14. What does state: present mean for a package?
Answer: Ensure that the package is installed.

15. What does state: started mean for a service?
Answer: Ensure that the service is running.

14. Quick Revision Sheet

Term	Remember
Playbook	YAML automation file.
Play	Defines target hosts and execution settings.
Task	Individual operation.
Module	Performs an operation.
Inventory	Defines managed hosts and groups.
hosts	Selects the targets.
become	Enables privilege escalation.
vars	Defines variables.
handlers	Tasks triggered by notifications.
notify	Notifies a handler when the notifying task reports a change.
--syntax-check	Checks syntax.
--check	Attempts check mode.
Idempotency	Repeated runs converge on the intended state.

15. Suggested Classroom Teaching Plan

Session	Topics	Suggested time
1	Definition, use cases, ad-hoc vs playbook	30 minutes
2	YAML syntax and playbook structure	35 minutes
3	First playbook and syntax-check	35 minutes
4	Apache installation and website deployment	45 minutes
5	Variables, handlers, idempotency	40 minutes
6	Troubleshooting and student exercises	45 minutes

Trainer tip: First explain the concept on the board, then demonstrate one command at a time, ask students to predict the result, execute it, and let students repeat the lab independently.